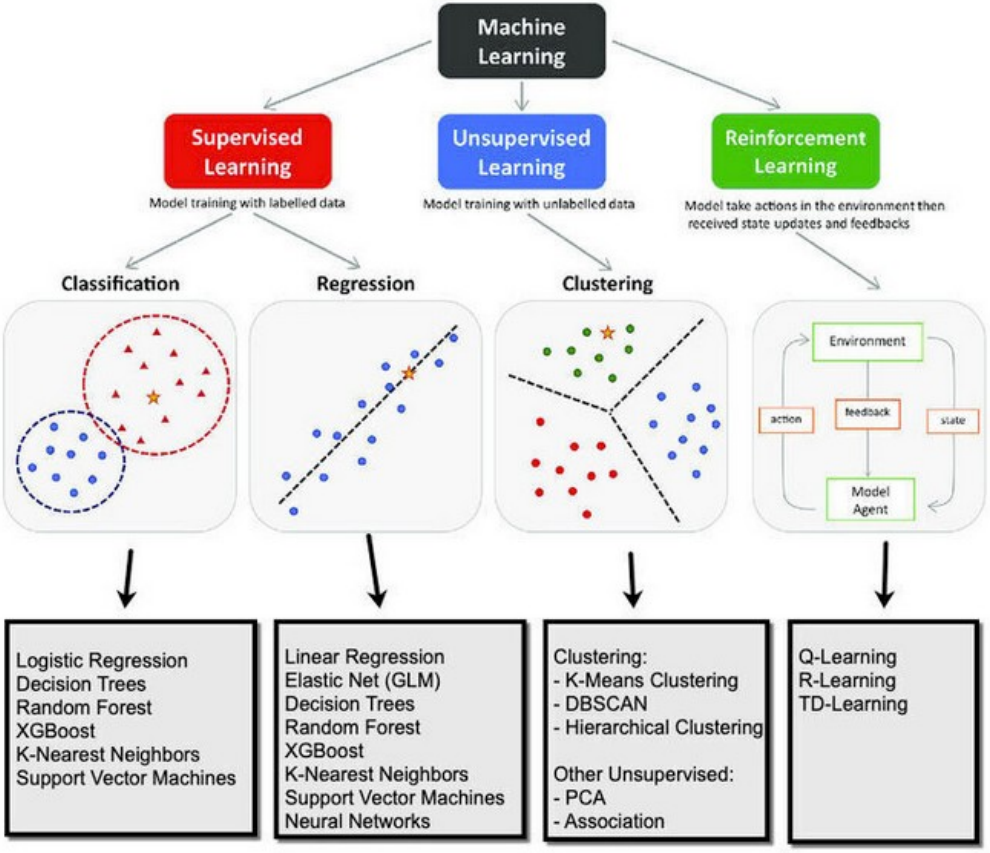
# Data-centric Machine Learning 07

Kyungjae Lee

# Machine Learning

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning



# Supervised vs. Unsupervised Learning

| 관점 | Supervised Learning | Unsupervised Learning |
|---|---|---|
| 입력 | Feature x 와 정답 y | Feature x |
| 목표 | 새 데이터의 정답 예측 | 데이터의 구조 / 패턴 탐색 |
| 오늘의 방법 | Decision Tree | K-means |
| 예시 | 학생의 합격 여부 예측 | 학습 패턴이 비슷한 학생 묶기 |

**지도학습은 정답을 이용하고 , 비지도학습은 정답 없이 유용한 구조 혹은 패턴을 찾는다**

## Decision Tree

어떤  feature 와 기준값으로
데이터를 나눌까 ?

분할 기준을 계산하고
과적합을 설명한다 .

## K-means

정답 없이 어떤 데이터를
같은 그룹으로 묶을까 ?

군집화 과정을 계산하고
거리의 역할을 설명한다 .

**같은 모델도 데이터의 품질 · 분포 · 표현에 따라 결과가 달라진다**

# Neural Network 와의 연결

| 방법 | 학습하는 것 | 학습을 진행하는 방식 |
|---|---|---|
| Neural Network | 예측 함수의 가중치 | Gradient descent |
| Decision Tree | Feature 와 분할 기준값 | 불순도 감소량 비교 |
| K-means | 군집의 중심점 | 할당과 평균 갱신 반복 |

**학습은 데이터에 맞는 규칙이나 구조를 찾는 과정이다**

# 비지도학습의 두 가지 목적

**같은 " 정답 없는 학습 " 이라도 무엇을 위해 배우는지에 따라 두 갈래로 나뉜다**

## ① 데이터를 이해하기 위한 분석

비슷한 표본을묶거나 저차원으로 표현

데이터 → 군집화 · 시각화 → 사람의 해석

**방법 :** Clustering, PCA, t-SNE

**결과 :** 고객 행동 군집 , 손글씨 숫자의 2 차원 시각화

## ② 모델을 위한 표현 학습 ( 자가지도 )

결과를 다른 모델 학습의 시작점으로 쓴다

대규모 데이터 → LLM·ViT 사전학습 → Checkpoint Embedding

**방법 :** 다음 토큰 예측 , 가려진 patch 복원

**결과 :** 좋은 starting point, 텍스트를 벡터변환하여 문서검색

# 비지도 학습에서 거리와 유사도는 어떻게 쓰일까 ?

**비지도 학습은 레이블 없이 데이터의 구조를 찾는다 .**

거리와 유사도는 데이터 간 관계를 나타내는 기준이다 .

## 군집화 (Clustering)

거리나 밀도 등을 기준으로 그룹을 찾는다 .

K-means, DBSCAN, 계층적 군집화

### 어떤 기준으로 묶는가 ?

K-means 는 각 점을 가장 가까운 중심에 배정한다 .

DBSCAN 은 점이 밀집해 연결된 영역을 찾는다 .

## 차원 축소 (Dimensionality Reduction)

데이터를 더 적은 수의 좌표로 표현한다 .

PCA, t-SNE

### 어떤 구조를 보존하는가 ?

PCA 는 분산이 큰 방향으로 투영한다 .

t-SNE 는 가까운 이웃 관계를 보존하려 한다 .

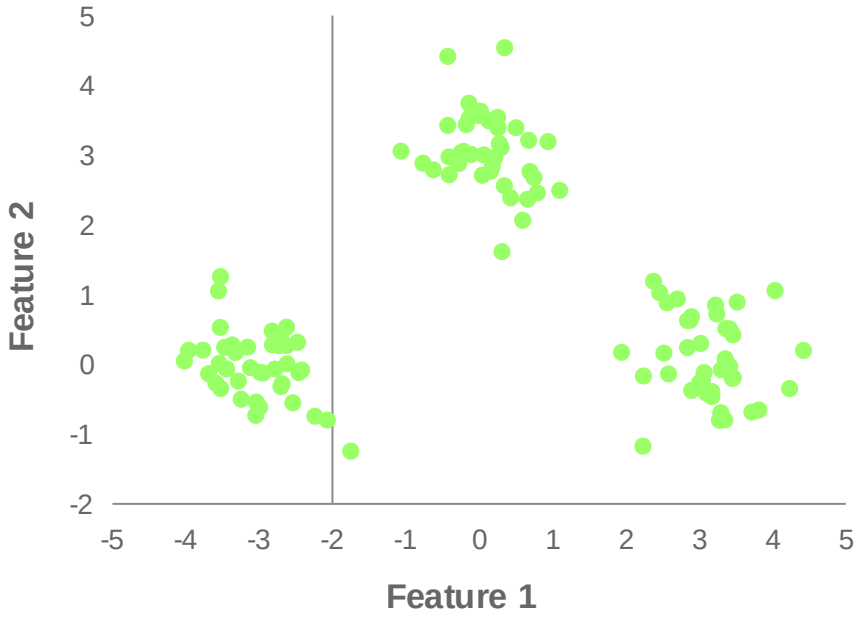
같은 표본이라도 Feature 혹은 Scale 을 바꾸면 거리와 분산이 달라질 수 있다 .

# Unsupervised Learning - ① Clustering:

**K-means:** 비슷한 데이터를 묶고

각 그룹의 중심을 찾는다

# Clustering: 정답 label 없이 그룹 찾기




**인풋에 label 이 없음**

Feature 사이의 거리를 이용해
비슷한 점을 같은 그룹에 넣는다 .

사람은 결과를 보고
각 그룹의 의미를 해석한다 .

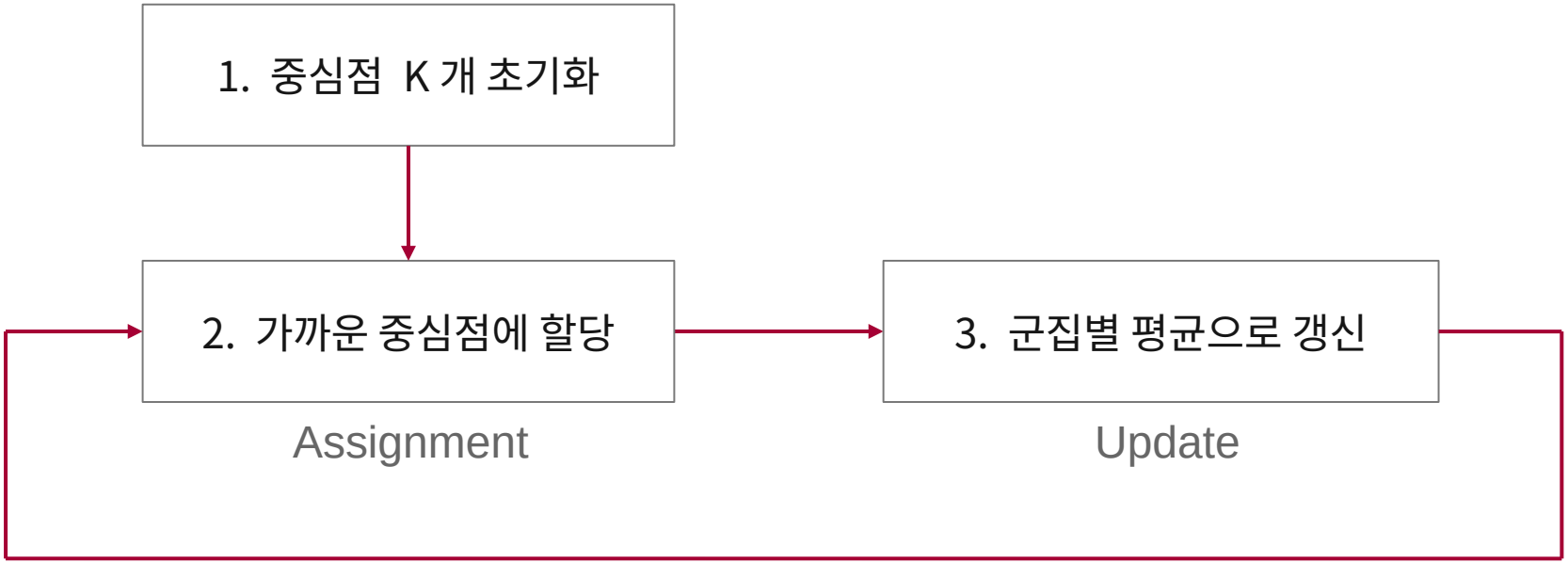
# K-means 의 목표

$$J(x, \mu) = \sum_i \left\| x^{(i)} - \mu_{c_i} \right\|^2$$

| 기호 | 의미 |
|---|---|
| K | 찾고자 하는 군집 수 |
| $\mu_j$ (centroid) | 군집 j 의 평균 위치 |
| $c_i$ | 데이터 i 에 할당한 군집 번호 |
| J (objective function) | 군집 안 거리 제곱합 |

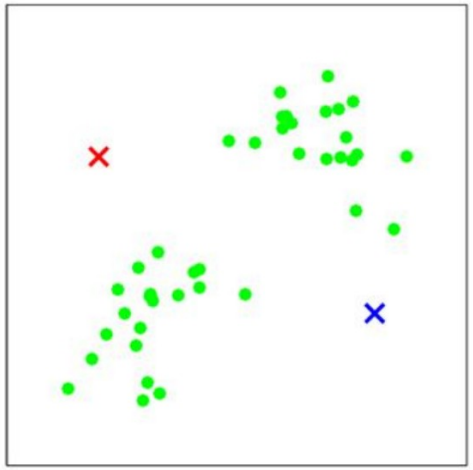
**같은 군집의 점들이 중심점에 가까울수록 objective function J 가 작다**

# K-means Algorithm




**할당과 평균 갱신을 반복하며 군집을 만든다**

# Step 0: 중심점 초기화



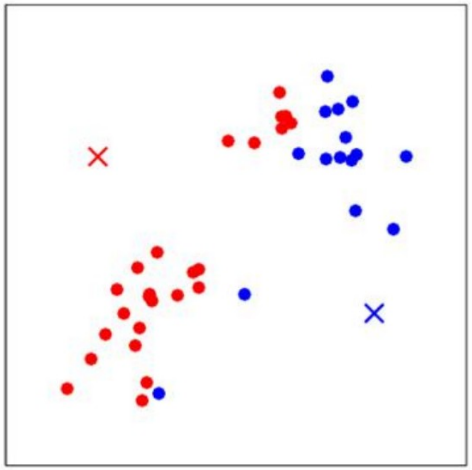
**K = 2**

(a) 입력 데이터 ( 초록 점 )
(b) 중심점 2 개를 임의 배치

× 표시 : 중심점
빨강 $\mu_1$ · 파랑 $\mu_2$

**초기 위치는 아직 최종 중심점이 아니다**

# Step 1: 가까운 중심점에 할당



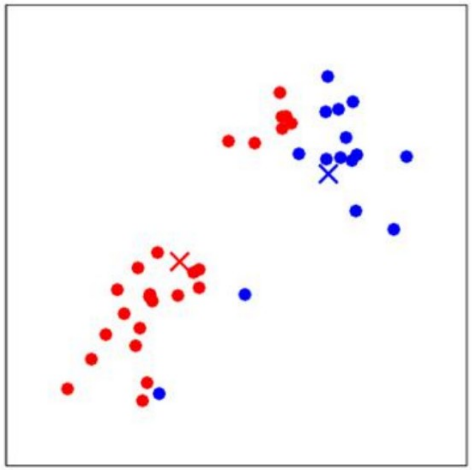
**각 점 → 더 가까운 ×**

$\| x - \mu_1 \|^2$ vs $\| x - \mu_2 \|^2$
더 작은 쪽 군집에 할당

위쪽 무리의 왼쪽 점들은
빨강 $\mu_1$ 에 더 가까워
빨강으로 할당된다

**중심점을 고정하고 각 데이터의 군집 번호를 정한다**

# Step 2: 평균으로 중심점 갱신



**새 중심점 = 군집의 평균**

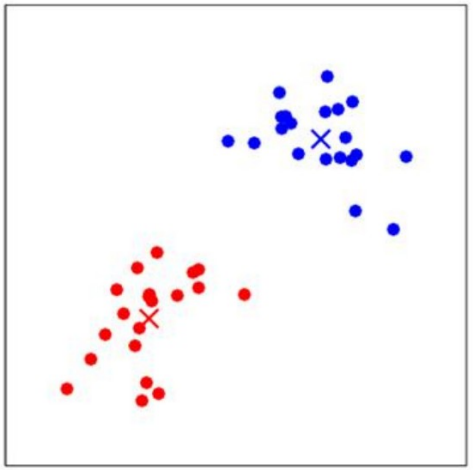
빨강 ×: 빨간 점들의 평균
→ 아래 무리 쪽으로 이동

파랑 ×: 파란 점들의 평균
→ 위 무리 쪽으로 이동

할당 ( 색 ) 은 그대로 유지

**할당 결과를 고정하고 중심점을 평균 위치로 이동한다**

# Repeat: 재할당과 수렴



| 단계 | 변화 |
|---|---|
| (e) 재할당 | 경계 점 색 변경 |
| (f) 갱신 | × → 무리 중앙 |
| 다음 반복 | 변화 없음 |

할당과 중심점이 더 이상
바뀌지 않으면 수렴

**J 는 증가하지 않지만 , 전역 최적해를 보장하지 않는다**

# Quiz: K-means  한 번 반복하기

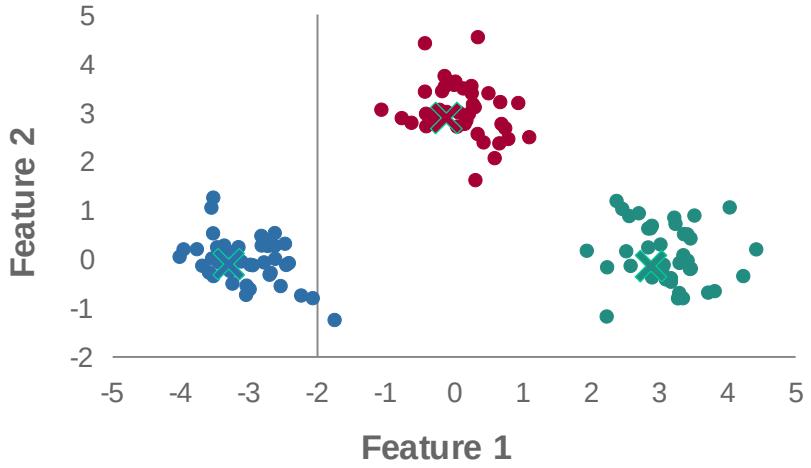
데이터 : 1, 2, 3, 8, 9, 10

**K = 2,  초기 중심점 : $\mu_1 = 2, \mu_2 = 8$**

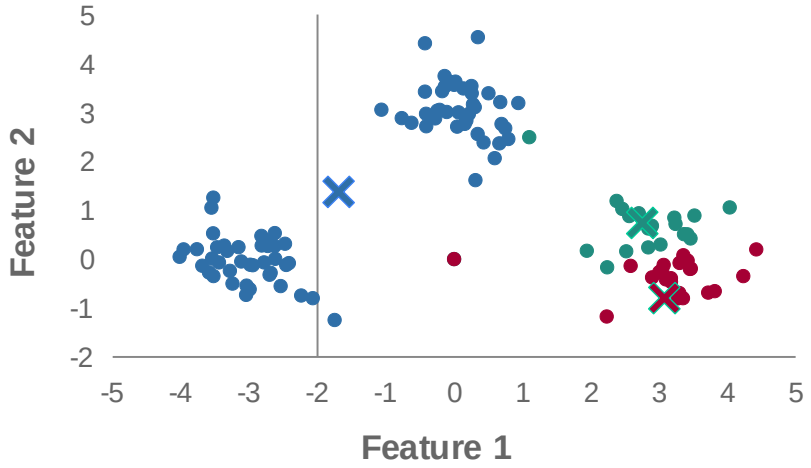
1. 각 데이터를 더 가까운 중심점에 할당하시오 .
2. 각 군집의 새 평균을 계산하시오 .
3. 갱신 후 거리 제곱합  J 를 계산하시오 .

# Issue: 초기화에 따른 결과 차이

**초기화 A: J = 66.12**




**초기화 B: J = 442.81**




**여러 초기값으로 실행하고 같은 조건에서 가장 작은 J 를 선택한다**

K-means++ 는 서로 떨어진 초기 중심점을 선택하도록 돕는다 .

# Feature Scaling: 단위가 거리를 바꾼다

**기준점 x = (0, 0)**

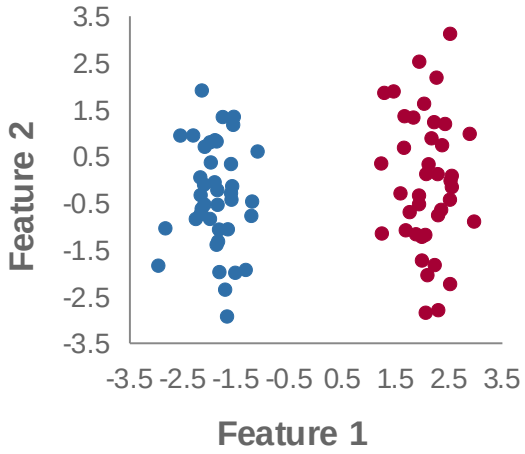
| 비교점 | 원래 좌표 | 원래 거리² | $x_2$ 를 100 배 | 변경 후 거리² |
|---|---|---|---|---|
| A | (2, 0) | 4 | (2, 0) | 4 |
| B | (0, 1) | 1 | (0, 100) | 10,000 |

단위를 바꾸기 전 : B 가 더 가깝다
단위를 바꾼 후 : A 가 더 가깝다

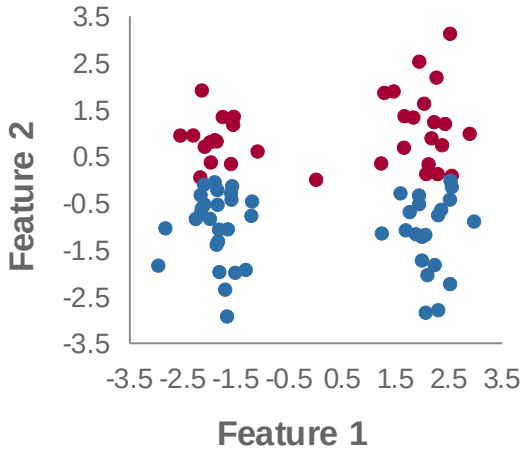
**한 feature 의 단위만 바꾸어도 군집 결과가 달라질 수 있다**

# Scaling 실험 : 같은 데이터 , 다른 군집

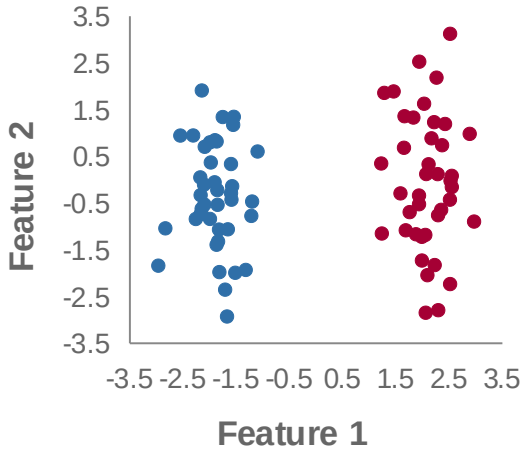
**원래 데이터**




**$x_2$ 만 100 배**




**표준화 후**




표준화 : z = (x − 평균 ) / 표준편차

모든 그림은 같은 원래 좌표로 표시 . 색은 각 조건의 군집 결과 .

# Outlier: 평균 중심점이 움직인다

| 데이터 | 평균 중심점 |
|---|---|
| 1, 2, 3 | μ = 2 |
| 1, 2, 3, 20 | μ = 6.5 |

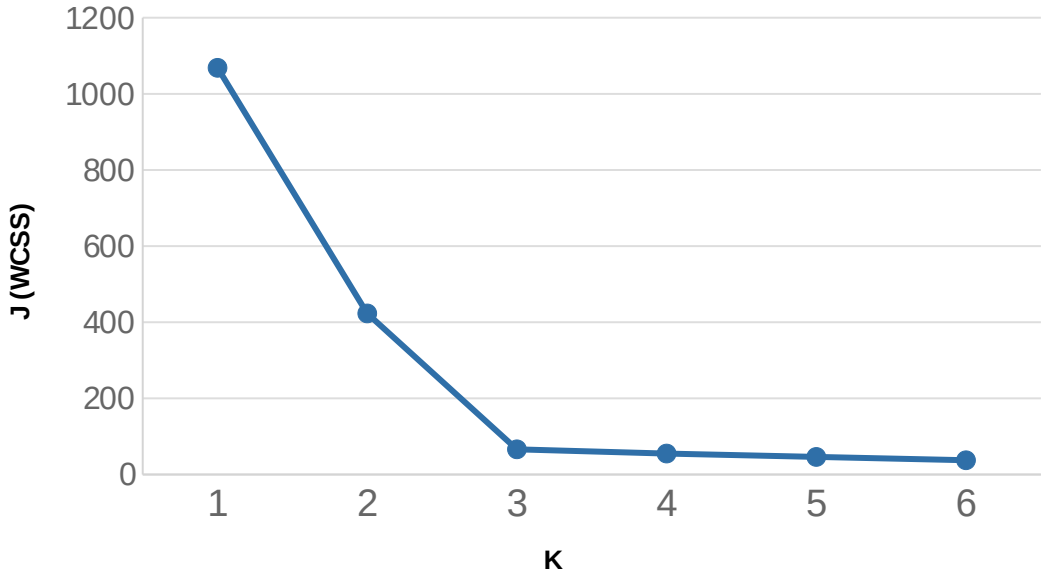
K = 1 인 예제 : 이상치 하나가 평균을 크게 이동시킨다 .

확인할 내용 : 실제 드문 사례인가 , 측정 · 입력 오류인가 ?

**이상치의 원인을 확인하고 데이터 처리 여부를 결정한다**

https://claude.ai/artifact/QshLtLDMktfe8Lh3hJz1p1?sk=_EDOq_pBrb3sh3CUJtxyzA

# K 선택 : Elbow Method




**이 예제에서는 K ≈ 3**

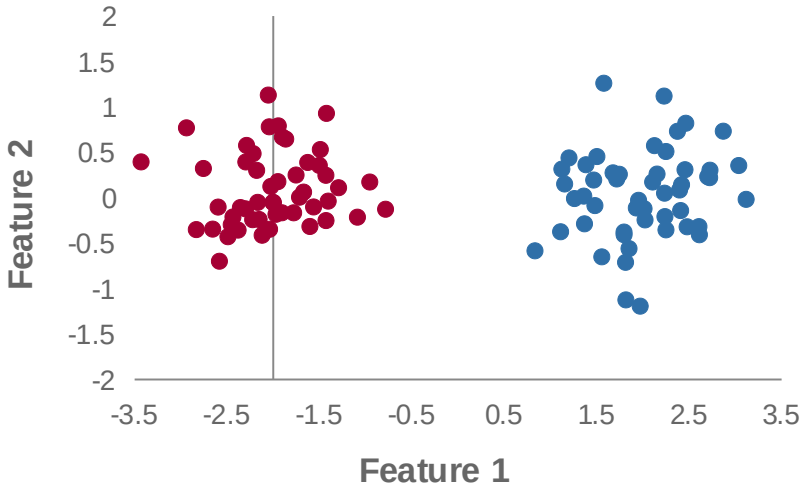
J 의 감소 폭이 작아지는
지점을 후보로 본다 .

그래프의 양 끝점을 잇는
직선에서 가장 멀리 떨어진 점의
K 을 선택한다 .

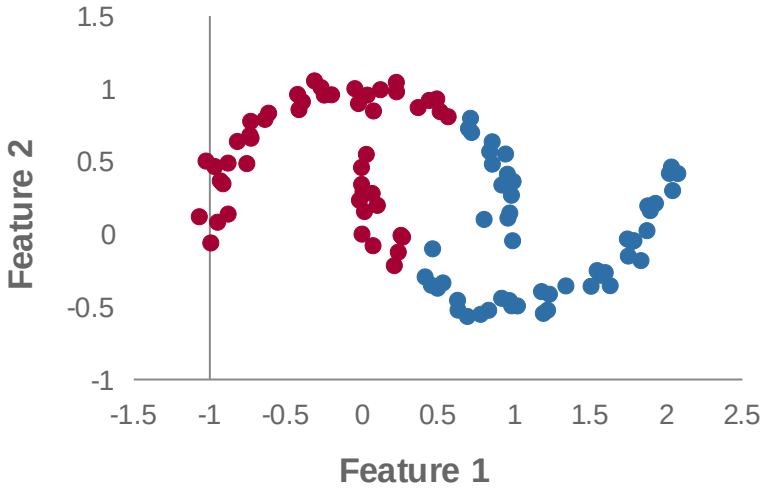
**K 를 계속 늘리면 J 는 0 으로 수렴하므로 , 가장 작은 J 만으로 K 를 고르지 않는다**

# Data Distribution 과 알고리즘의 가정

## 잘 분리된 원형 군집

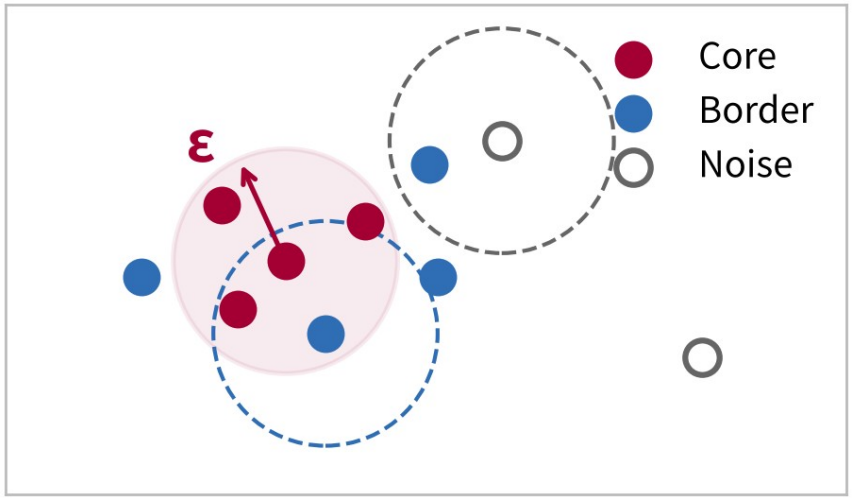



## 곡선 형태 : Two Moons




**K-means 는 원형에 가까운 군집에 잘 맞으며 , Two Moons 구조에선 잘 나누기 어렵다**

# DBSCAN: 거리 대신 밀도로 묶기




예 : minPts = 4 ( 자기 자신 포함 ), ε = 빨간 원의 반경

## 두 개의 기준값

**ε (epsilon)**: 이웃으로 볼 반경

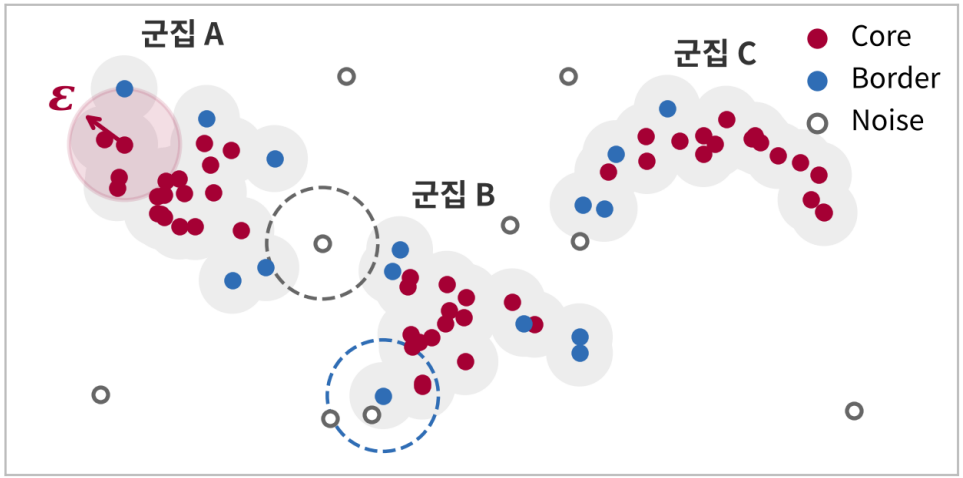
**minPts**: 반경 **ε** 안에 점이 최소 몇 개 있어야 밀집하다고 볼 것인가

**Core**: ε 안에 이웃이 minPts 개 이상

**Border**: core 근처 , 이웃은 부족

**Noise**: 어느 core 와도 닿지 않음

# DBSCAN: 거리 대신 밀도로 묶기

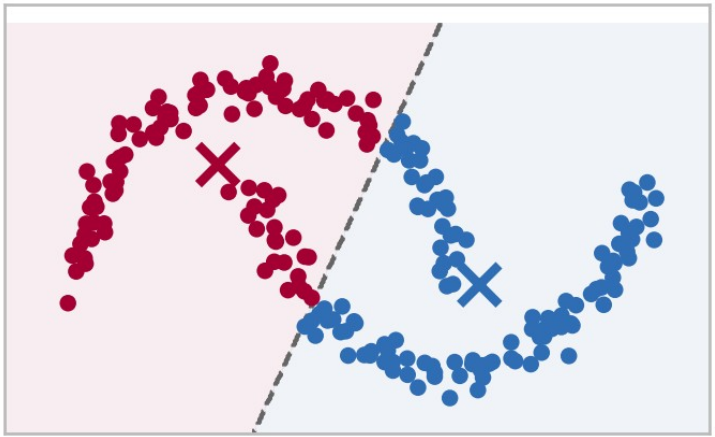



1. **점의 유형 판정 :** 핵심점 (Core), 경계점 (Border), 잡음점 (Noise) 분류
2. **핵심점 (Core) 연결 :** 거리가 ε 이하인 핵심점끼리 잇고 , 연결된 핵심점들을 같은 군집으로 묶는다 .
3. **경계점 (Border) 배정 :** 경계점을 이웃 핵심점의 군집에 포함한다 . 두 군집에 닿으면 탐색 순서에 따라 소속이 달라질 수 있다 .

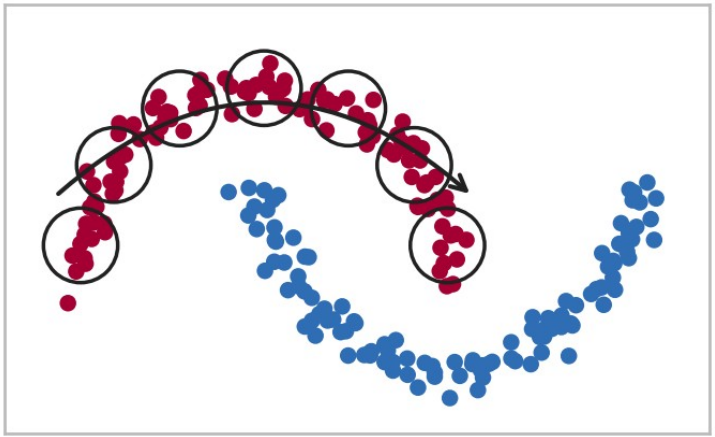
**중심점 대신 , 각 점 주변이 얼마나 빽빽한지 ( 밀도 ) 를 본다**

# DBSCAN 은 왜 두 초승달을 나눌 수 있을까

**K-means: 어느 중심이 가까운가 ?**



**DBSCAN: 이웃이 이어지는가 ?**



**① 중심 대신 이웃**
반경 ε 안의 이웃만 본다
직선 경계를 긋지 않는다

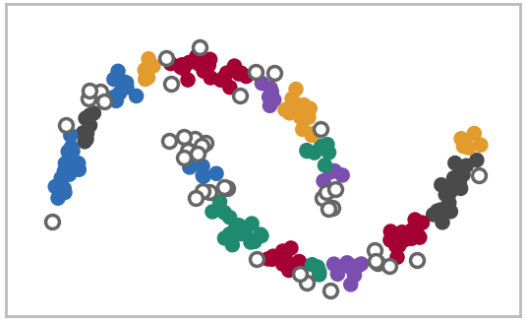
**② 연결 (chaining)**
핵심점끼리 이어지면 같은 군집
곡선을 따라 계속 확장

**③ 빈 공간 = 경계**
초승달 사이는 밀도가 낮아
사슬이 끊긴다

**빽빽한 이웃의 사슬이 곡선을 따라 이어지면 하나의 군집이 된다**

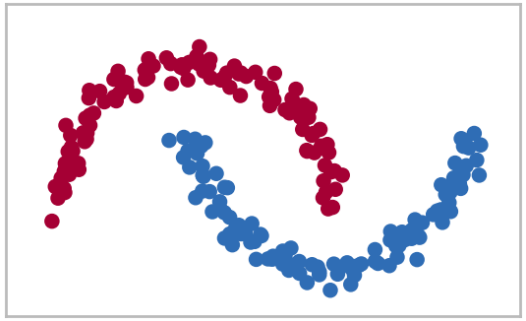
# DBSCAN 의 데이터 요소 : ε 가 밀도의 기준

**ε 너무 작음 (0.1)**



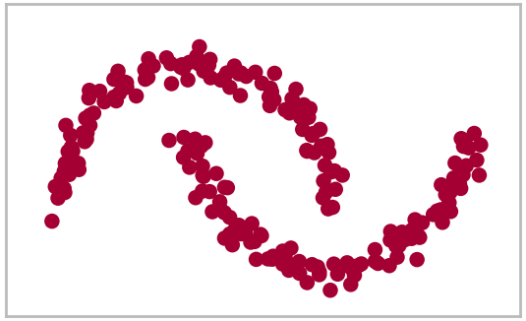
대부분 noise, 작은 조각 다수

**ε 적절 (0.2)**



두 초승달 분리 , noise 없음

**ε 너무 큼 (0.6)**



두 초승달이 하나로 합쳐짐

**Scale** ε 는 거리 단위이므로 feature scaling 이 결과를 바꾼다
**이상치** 억지로 군집에 넣지 않고 noise 로 남긴다
**밀도 차이** 밀도가 서로 다른 군집은 하나의 ε 로 나누기 어렵다

**K 는 정하지 않지만 , 밀도의 기준 ε 와 minPts 는 정해야 한다**

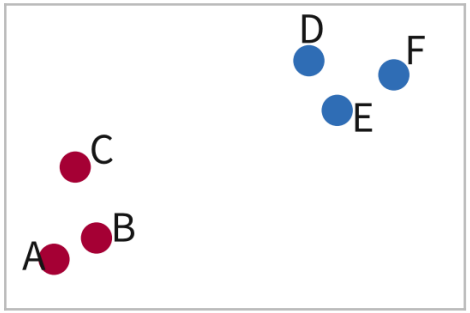
# Agglomerative Hierarchical Clustering

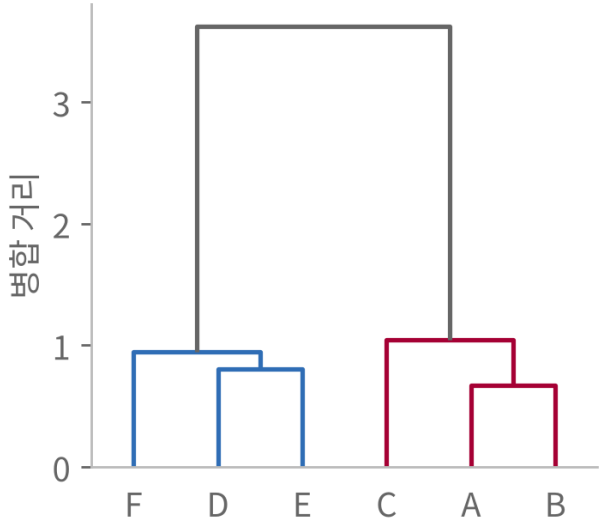
**K 나 ε 하나를 고르지 않고 , 모든 수준의 군집을 한 번에 볼 수는 없을까 ?**

1. 모든 점이 각자 하나의 군집
2. 가장 가까운 두 군집을 병합
3. 하나가 남을 때까지 반복

병합 순서와 거리를 트리로
**기록 → Dendrogram**

예 : A–B → D–E → F → C → 전체




**가까운 군집부터 차례로 합치고 , 병합 순서를 트리로 기록한다**

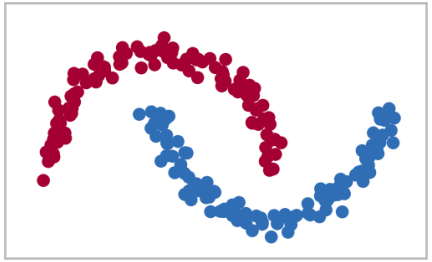
# Linkage: 군집 사이의 거리를 정의한다

| Linkage | 두 군집 사이의 거리 |
|---|---|
| Single | 가장 가까운 두 점의 거리 |
| Complete | 가장 먼 두 점의 거리 |
| Average | 모든 점 쌍의 평균 거리 |
| Ward | 합쳤을 때 $J$ 의 증가량 |

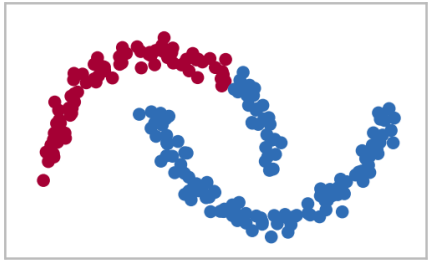
**Single** ≈ DBSCAN 처럼 연결을 따라간다

**Ward** ≈ K-means 처럼 $J$ 를 줄인다

**Single: 두 초승달 분리**



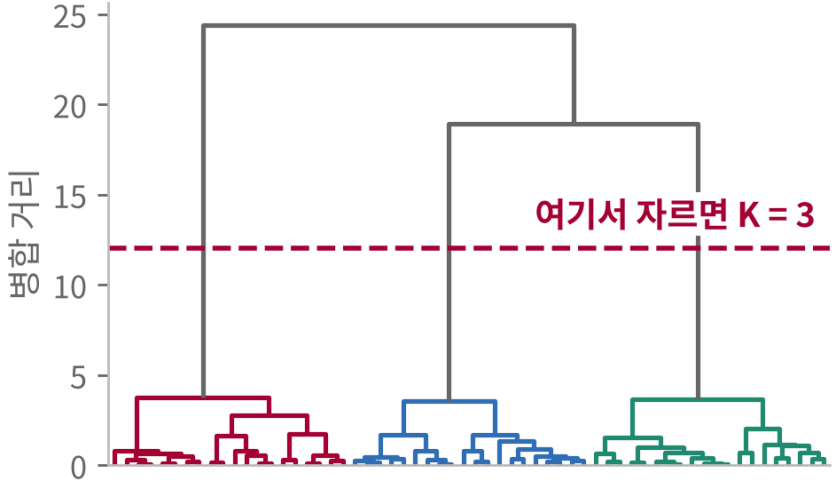
**Ward: 초승달 끝이 잘못 묶임**



**Linkage 를 바꾸면 같은 데이터에서도 군집의 모양이 달라진다**

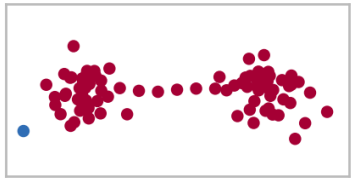
# Dendrogram 자르기와 데이터 요소

## 자르는 높이가 K 를 정한다

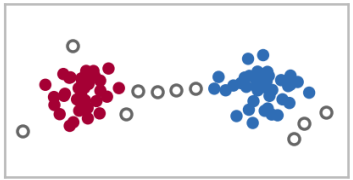



세로 간격이 크게 벌어지는 곳 = 자연스러운 K 후보

## Single linkage 의 약점 : chaining



Single: 하나로 연결



DBSCAN: 다리는 noise

**Scale** 거리 기반 → 표준화가 중요

**이상치** 혼자 따로 군집이 되기 쉽다

**비용** 모든 쌍의 거리 → 큰 데이터는 느림

**Dendrogram 은 K 를 미리 정하지 않고 여러 수준의 군집을 보여준다**

# 세 가지 군집화 방법 비교

| 관점 | K-means | DBSCAN | Hierarchical |
|---|---|---|---|
| 군집의 기준 | 중심점까지의 거리 | 이웃 밀도의 연결 | 군집 사이 거리 (linkage) |
| 미리 정할 값 | K | ε, minPts | Linkage, 자를 높이 |
| 잘 맞는 형태 | 원형 , 비슷한 크기 | 임의 모양 , 비슷한 밀도 | Linkage 에 따라 다름 |
| 이상치 | 평균을 끌어당김 | Noise 로 분리 | 단독 군집이 되기 쉬움 |
| Two moons | 실패 | 성공 | Single 성공 , Ward 일부 실패 |

**알고리즘의 가정이 데이터의 형태 · 밀도 · 스케일과 맞는지 먼저 확인한다**

# Unsupervised Learning - ②차원 축소 / 시각화

**PCA & t-SNE** : 고차원 데이터를 줄여

구조를 눈으로 확인한다

# 왜 차원을 줄이는가

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 7 | 15 | 13 | 1 | 0 | 0 |
| 0 | 8 | 13 | 6 | 15 | 4 | 0 | 0 |
| 0 | 2 | 1 | 13 | 13 | 0 | 0 | 0 |
| 0 | 0 | 2 | 15 | 11 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 12 | 12 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 10 | 8 | 0 |
| 0 | 0 | 8 | 4 | 5 | 14 | 9 | 0 |
| 0 | 0 | 7 | 13 | 13 | 9 | 0 | 0 |

손글씨 숫자 한 장 = 8×8 픽셀
sklearn digits, 1,797 장

## 한 이미지 = 64 차원 벡터

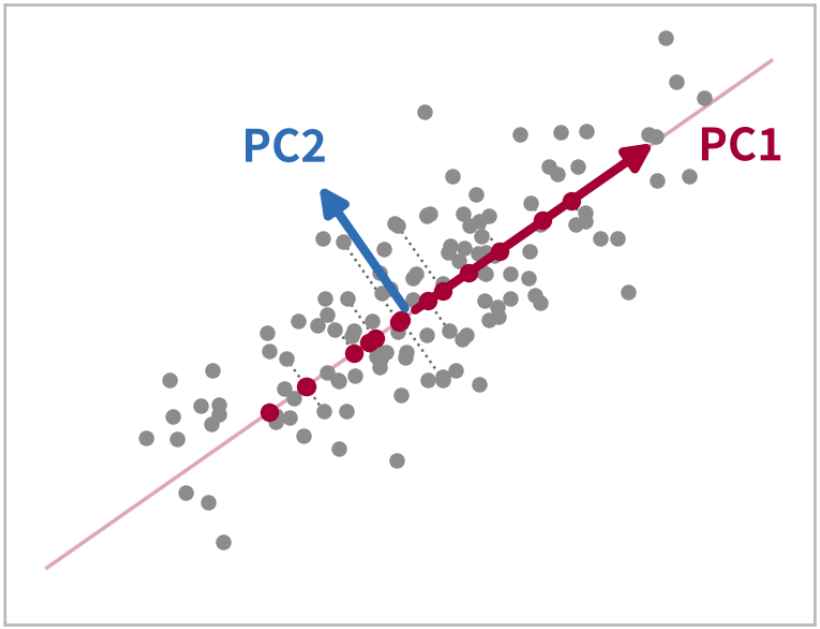
사람은 2~3 차원까지만 눈으로 볼 수 있다
학생 기록 , 센서 , 문서도 feature 가 수십 ~ 수천 개

## 차원 축소로 하는 일

**구조 확인** : 군집이 실제로 나뉘어 있는가 ?
**데이터 점검** : 이상치 , label 오류 후보 찾기
**효율** : 노이즈와 계산량 줄이기

**고차원 데이터의 중요한 구조를 유지하면서 저차원으로 축소한다**

# Principal Component Analysis (PCA)




**PC1**: 분산이 가장 큰 방향

**PC2**: PC1 과 직교 , 남은 분산이 가장 큰 방향

## 투영 (projection)

z = (x − 평균 ) · w

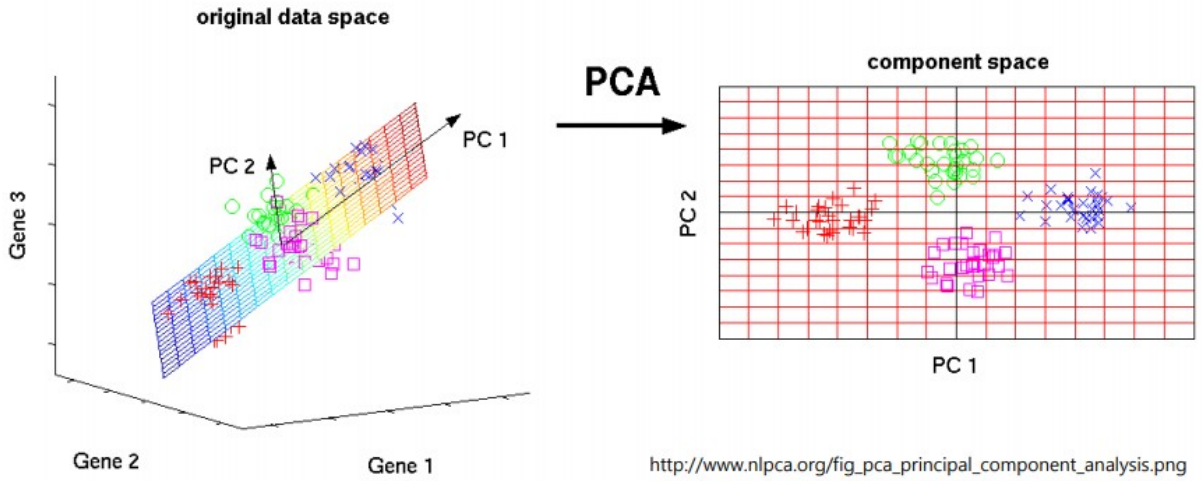
빨간 점 : 각 점을 PC1 위로 내린 위치

**이 예제 : PC1 이 전체 분산의 90.8%**

→ 2 차원을 1 차원으로 줄여도 대부분 유지

**정보 ( 분산 ) 를 가장 많이 남기는 축만 골라 차원을 줄인다**

# Principal Component Analysis (PCA)




http://www.nlpca.org/fig_pca_principal_component_analysis.png

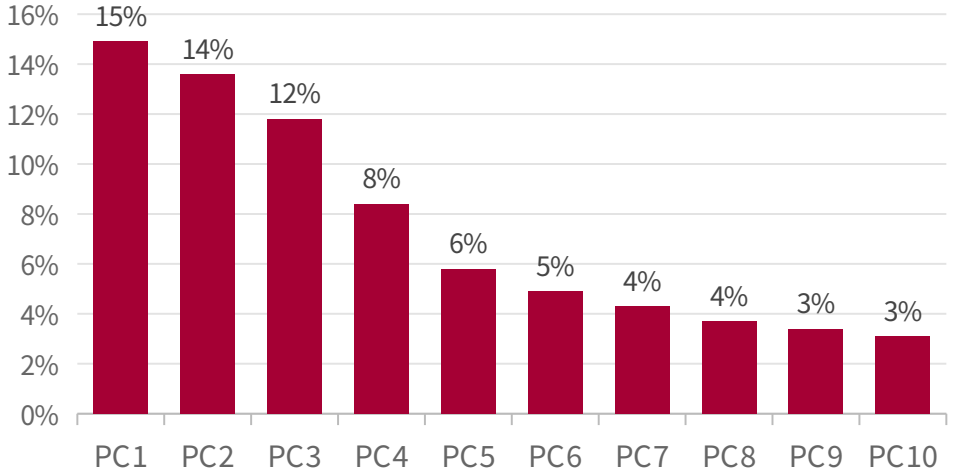
- Finds k directions in which data has highest variance
  - Principal directions (eigenvectors) $W$
- Projecting inputs $x$ on these vectors yields reduced dimension representation (&decorrelated)
  - Principal components
  - $h = f_\theta(x) = W(x - \mu)\ with\ \theta = \{W, \mu\}$

- Why mention PCA?
  - Prototypical unsupervised representation learning algorithm
  - Related to autoencoders
  - Prototypical manifold modeling algorithm

# PCA 계산 과정과 설명된 분산

1. 중심화 : 각 feature 의 평균을 뺀다
2. 공분산 행렬 Σ 를 계산한다
3. Σ 의 고유벡터 = 주성분 방향
   고유값 = 그 방향의 분산
4. 고유값이 큰 순서로 k 개를 골라
   그 축 위로 투영한다

**digits (64 차원 ): 주성분별 설명된 분산 비율**

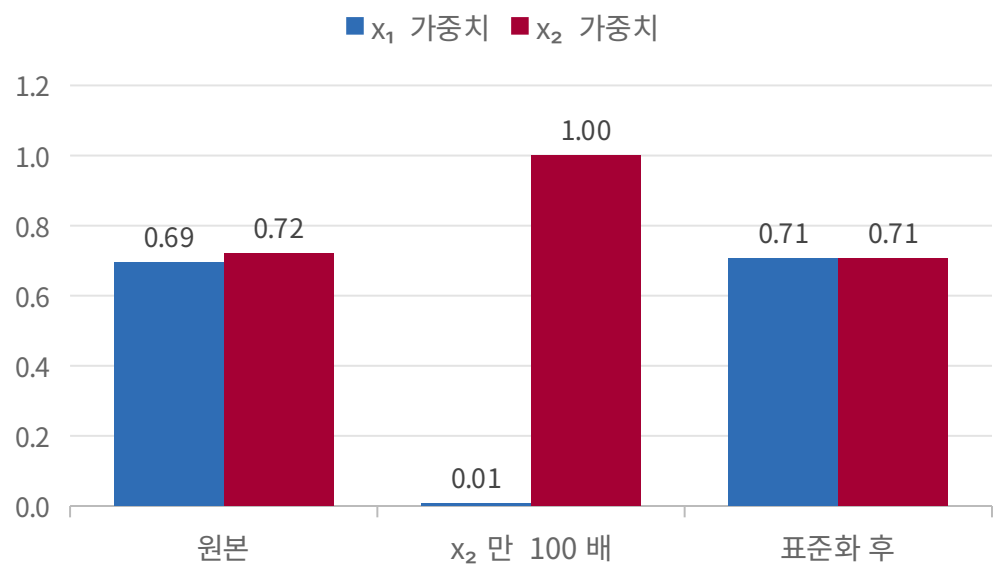



PC1+PC2 = **28.5%** · 90% 를 설명하려면 **21 개**

**남길 축의 수는 설명된 분산 비율을 보고 정한다**

# PCA 와 Scaling: 단위가 축을 바꾼다

**PC1 방향의 가중치 |w| ( 상관계수 0.8 인 두 feature)**




## 같은 데이터 , 다른 PC1

**원본** : $x_1$, $x_2$ 를 비슷하게 사용

**$x_2$ 만 100 배** : PC1 ≈ $x_2$ 축

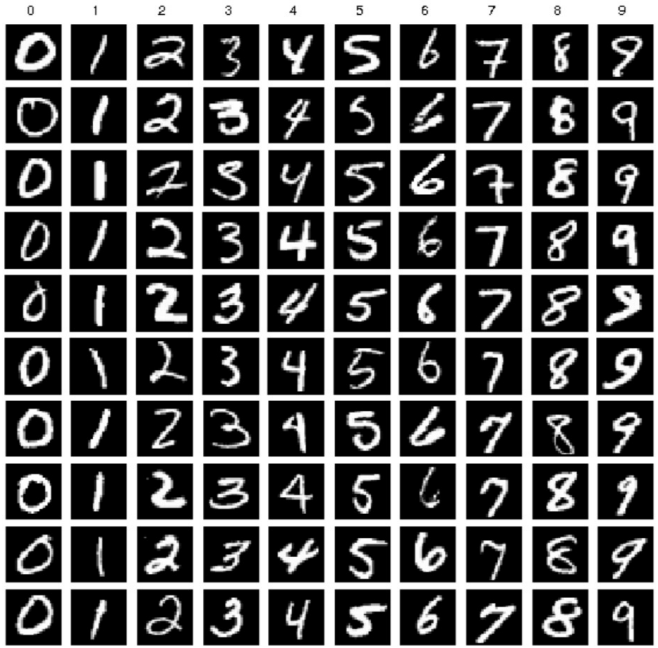
**표준화 후** : 다시 균형

PCA 는 분산을 최대화한다
→ 단위가 큰 feature 의 분산이 PC1 을 결정한다

**K-means 와 마찬가지로 , PCA 전에 feature 의 단위를 확인한다**

# PCA 의 한계 : 가까운 이웃이 섞인다

**sklearn digits** (8 × 8 픽셀 = 64 차원 )




**PCA 2 차원 축소**



# PCA 의 한계 : 가까운 이웃이 섞인다




## digits 64 차원 → PC1, PC2

0, 4, 6 은 비교적 떨어져 있지만
2, 3, 5, 8, 9 등은 크게 겹친다

## 왜 그럴까 ?

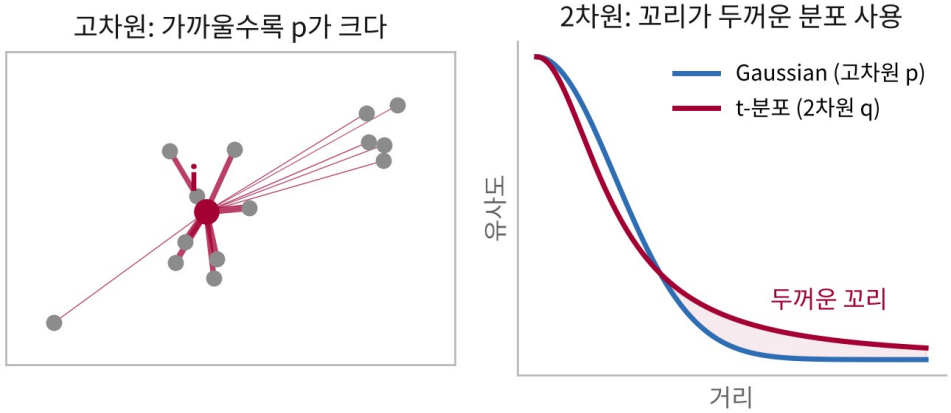
PCA 는 직선 축으로의 선형 투영
전체 분산 ( 전역 ) 은 최대한 보존하지만
가까운 이웃 ( 국소 ) 관계는 섞일 수 있다

점의 색과 숫자는 해석용 label 이며 학습에는 쓰지 않았다

**PCA 는 전역 분산을 보존하지만 , 국소 이웃 관계는 놓칠 수 있다**

# t-SNE: 가까운 이웃 관계를 보존한다

1. **고차원에서 이웃 확률 p 계산**
   가까울수록 큰 값
   이웃의 범위는 perplexity 로 조절
2. **2 차원에 점을 놓고 q 계산**
   꼬리가 두꺼운 t- 분포를 사용
3. **p 와 q 가 같아지도록 점을 이동**
   KL divergence 를
   gradient descent 로 줄인다




두꺼운 꼬리 : 2 차원에서 덜 가까운 점을 더 멀리 둘 수 있어
서로 다른 무리 사이가 잘 벌어진다

**t-SNE 는 고차원에서 가까운 이웃을 저차원에서도 보존하려 한다**

# PCA vs t-SNE: 같은 digits 데이터

**PCA ( 선형 , 전역 )**




**t-SNE ( 비선형 , 국소 )**

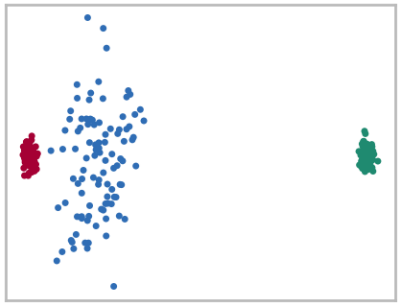



점의 색과 숫자는 실제 label 로 , 해석을 위해서만 표시했다 ( 두 방법 모두 학습에 label 을 쓰지 않음 )

**t-SNE 에서는 숫자별 무리가 뚜렷하게 드러난다**

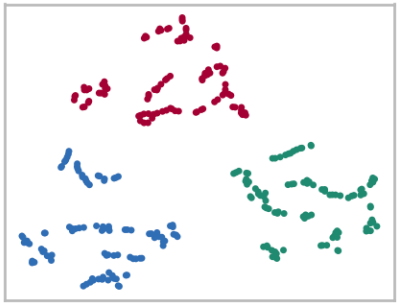
# t-SNE 의 데이터 요소 : 해석할 때 주의할 점

**원본 (2 차원 )**



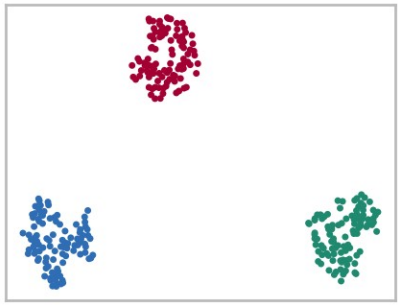
파랑은 넓게 , 초록은 멀리

**perplexity 5**



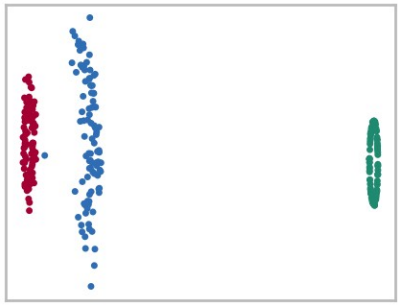
작은 조각으로 쪼개짐

**perplexity 30**



크기 · 간격이 비슷해짐

**perplexity 100**



원래 배치에 더 가까움

**크기** : 무리의 퍼짐 정도는 의미가 없다

**거리** : 무리 사이 거리도 그대로 믿기 어렵다

**설정** : perplexity·seed 를 바꿔 여러 번 확인

**변환** : 새 데이터를 같은 지도에 넣기 어렵다

**t-SNE 그림은 가설을 세우는 도구이지 , 결론이 아니다**

# 실전 흐름 : 군집화와 시각화를 함께 쓰기

① **표준화**
단위 맞추기 ➡ ② **PCA**
노이즈 · 차원 축소 ➡ ③ **군집화**
원래 ·PCA 공간 ➡ ④ **t-SNE**
2 차원으로 보기 ➡ ⑤ **해석 · 점검**
이상치 , label 오류

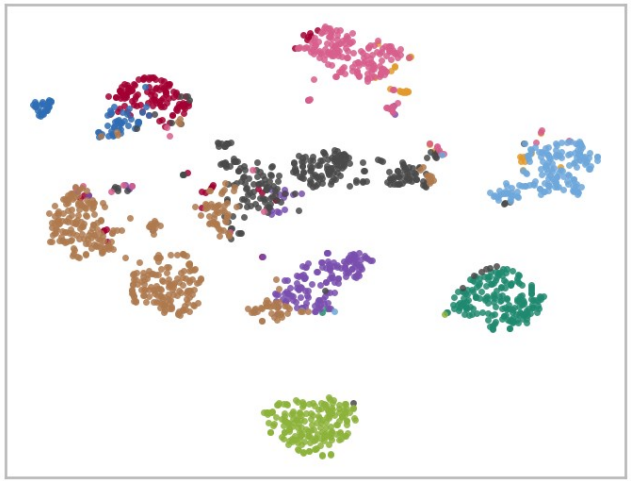


**그림 : t-SNE 지도 위에 K-means 결과 (K=10)**

군집은 표준화 → PCA 30 차원 공간에서 계산
한 무리 안에 색이 섞인 곳 → 다시 확인할 후보

**피해야 할 것**

t-SNE 좌표 위에서 바로 군집화하기
→ 크기 · 거리가 왜곡된 공간이다

**군집은 원래의 Feature 공간에서 계산하고 , t-SNE 는 결과를 보는 창으로 쓴다**

# 정리 : 전역 기준 vs 국소 기준

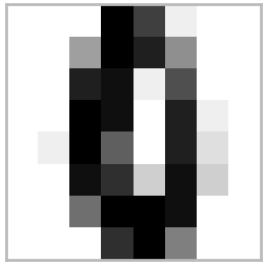
| 관점 | 전역 기준 (global) | 국소 기준 (local) |
|---|---|---|
| 무엇을 보나 | 전체 평균 · 분산 · 중심 | 가까운 이웃 · 밀도 |
| Clustering | K-means, Ward | DBSCAN, Single linkage |
| 시각화 | PCA | t-SNE |
| 강점 | 단순하고 빠르며 해석이 쉽다 | 휘어진 구조 , 임의의 모양 |
| 주의 | 비선형 구조를 놓친다 | 설정에 민감 , 크기 · 거리 해석 주의 |

**데이터의 모양에 맞는 기준을 고르고 , 여러 방법으로 교차 확인한다**

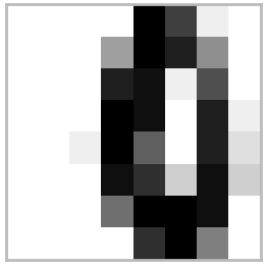
# 거리 자체를 학습한다면 ?

**오늘의 방법론은 feature 공간의 유클리드 거리에 기반함**

원본 숫자 0

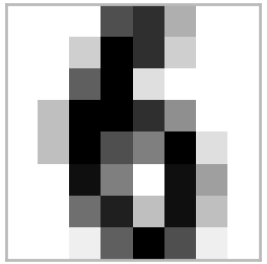


한 칸 오른쪽 이동



**원본과 거리 63.9**

다른 숫자 6



**원본과 거리 27.3**

## 픽셀 거리 ≠ 의미 거리

같은 0 을 한 칸 옮겼을 뿐인데
다른 숫자 6 보다 더 멀다

## 8 강 : 표현 학습

Autoencoder · Masked prediction
Contrastive learning · Embedding model

→ **의미가 비슷하면 가까운 공간을 학습**

**데이터를 더 잘 표상하는 Feature 가 있다면 , 오늘 배운 방법이 훨씬 잘 작동한다**

# E N D